Project

COAL LAB

SUBMITTED BY

Sabiha Liaqat

BSDSM-F23-006

BSDS-6A

SE DEPARTMENT

SUBMITTED BY

SIR ADEEL

Project Title:

Student Result Anlayzer

Introduction

The Student Result Management System is a simple console-based application developed using 16-bit Assembly Language. The system is designed to take marks of three subjects, perform calculations, and display the final result including total marks, percentage, grade, and pass/fail status.

This project demonstrates how low-level programming can be used to perform arithmetic operations, handle input/output using interrupts, and implement decision-making logic.

Objective

The main objective of this project is to:

- Understand the basics of Assembly Language programming
- Learn how to handle input and output using DOS interrupts
- Perform arithmetic operations at a low level
- Implement conditional logic for grading and results
- Develop a simple and efficient result calculation system

Tools and Technologies

- NASM (Netwide Assembler)
- DOSBox Emulator
- 16-bit x86 Assembly Language
- INT 21h DOS Interrupts

System Overview

This program works in a step-by-step process:

The program starts execution in DOS environment.

The user is prompted to enter marks for three subjects:

- English

- Math
- Computer

The program reads and processes numeric input.

It calculates:

- Total Marks
- Percentage

Based on the percentage, a grade is assigned.

The system also determines whether the student has passed or failed.

Finally, all results are displayed on the screen.

```
bits 16
org 100h

jmp start

;  DATA SECTION
msg1 db 10,13,"Enter English (0-100): $"
msg2 db 10,13,"Enter Math (0-100): $"
msg3 db 10,13,"Enter Computer (0-100): $"

totalmsg db 10,13,"Total: $"
permsg   db 10,13,"Percentage: $"
grademsg db 10,13,"Grade: $"
passmsg  db 10,13,"Result: $"

m1 dw 0
m2 dw 0
m3 dw 0
total dw 0
per dw 0


start:
```

```
; Data segments setup
mov ax, cs
mov ds, ax

;  Input English
mov dx, msg1
mov ah, 09h
int 21h
call input
mov [m1], ax

 Input Math
mov dx, msg2
mov ah, 09h
int 21h
call input
mov [m2], ax

; Input Computer
mov dx, msg3
mov ah, 09h
int 21h
call input
mov [m3], ax

;Calculate Total
mov ax, [m1]
add ax, [m2]
add ax, [m3]
mov [total], ax
```

```
; Show Total
mov dx, totalmsg
mov ah, 09h
int 21h
mov ax, [total]
call print_num


mov ax, [total]
mov bx, 3
xor dx, dx
div bx
mov [per], ax

; Show Percentage
mov dx, permsg
mov ah, 09h
int 21h
mov ax, [per]
call print_num

;  Grade Logic
mov dx, grademsg
mov ah, 09h
int 21h

mov ax, [per]
cmp ax, 80
jae label_A
cmp ax, 60
jae label_B
cmp ax, 40
jae label_C
jmp label_F
```

```
label_A:
    mov dl, 'A'
    jmp show_g
label_B:
    mov dl, 'B'
    jmp show_g
label_C:
    mov dl, 'C'
    jmp show_g
label_F:
    mov dl, 'F'

show_g:
    mov ah, 02h
    int 21h

    ;  Result Pass/Fail
    mov dx, passmsg
    mov ah, 09h
    int 21h
    mov ax, [per]
    cmp ax, 40
    jl is_fail
    mov dl, 'P'
    jmp disp_res
is_fail:
    mov dl, 'F'
disp_res:
    mov ah, 02h
    int 21h
```

```
    ; Final Exit
    mov ah, 4Ch
    int 21h




input:
    xor bx, bx
next_digit:
    mov ah, 01h
    int 21h
    cmp al, 13
    je input_done
    sub al, '0'
    mov ah, 0
    mov cx, ax
    mov ax, bx
    mov dx, 10
    mul dx
    add ax, cx
    mov bx, ax
    jmp next_digit
input_done:
    mov ax, bx
    ret

; Print Number Function
print_num:
    push ax
    push bx
    push cx
    push dx
```

```
; Print Number Function
print_num:
    push ax
    push bx
    push cx
    push dx

    mov bx, 10
    xor cx, cx


    test ax, ax
    jnz convert_loop
    mov dl, '0'
    mov ah, 02h
    int 21h
    jmp print_exit

convert_loop:
    xor dx, dx
    div bx
    push dx
    inc cx
    test ax, ax
    jnz convert_loop

display_loop:
    pop dx
    add dl, '0'
    mov ah, 02h
    int 21h
    loop display_loop
```

```
print_exit:
    pop dx
    pop cx
    pop bx
    pop ax
    ret
```

Working Logic

Input Module

The system takes numeric input from the user digit by digit and converts it into a complete number using a custom input function.

Calculation Module

- Total Marks = English + Math + Computer
- Percentage = Total Marks / 3

Grading System

- 80 – 100 → Grade A
- 60 – 79 → Grade B
- 40 – 59 → Grade C
- Below 40 → Grade F

Result Decision

- If Percentage ≥ 40 → Pass
- Otherwise → Fail

Features of the System

- Accepts user input for multiple subjects
- Handles multi-digit number input
- Performs arithmetic operations (addition and division)
- Implements conditional branching for grading
- Displays formatted output using DOS interrupts
- Lightweight and fast execution

Implementation Details

The program is written in 16-bit Assembly Language and runs in DOSBox. It uses INT 21h services for input and output operations.

Key procedures include:

- Input procedure for reading numbers
- Calculation logic for total and percentage
- Print procedure for displaying numeric output
- Grade evaluation using CMP and conditional jumps

Conclusion

This project helped in understanding the fundamentals of Assembly Language programming. It provided practical knowledge of how low-level systems handle input/output, perform calculations, and make decisions.

Overall, the Student Result Management System is a simple but effective project that demonstrates core concepts of system-level programming.

GITHUB LINK :
https://github.com/sabiha9116/COAL-project-report.git